Test Report — Topics get their own table

PR: kuantorflow#209 (+ kuantorflow_automation, this branch) · **Issue:** kuantorflow#207 · **Date:** 2026-08-04

Summary

Topics moved from a VARCHAR on `flashcards` to a table of their own, with `flashcards.topic_id` pointing at it and `topics.created_by_user_id` recording who started each one. `apply_schema.py` grew a third kind of step to carry the backfill. Verified against a clone of the live local database and on purpose-built scratch databases: **626 tests pass, none failing** (575 before), and the migration is idempotent, resumable, and invisible on the page.

What changed

- **`schema.sql`** — new `topics` table (`name` UNIQUE, `created_by_user_id`, `created_at`), placed **above** `flashcards` because the file is applied in order and a foreign key needs its target first. `flashcards` gains `topic_id`, `idx_topic_id` and `fk_flashcards_topic`.
- **`apply_schema.py`** — a `Pending(probe)` step target for data migrations, plus the four steps that take an existing database to #207: the column, the backfill, the index, the key.
- **`utils.py`** — `_get_or_create_topic()` is the single place a topic name becomes an id; `save_flashcard()` and `move_flashcard()` write both `topic` and `topic_id`; `get_topics()` and `get_flashcards_by_topic()` join `topics`; `_owner_clause()` qualifies its column now that a join is involved.
- **`applog.py`** — `topic_created()`, writing a `TOPIC` line to `cards.log`.
- **`CLAUDE.md`** — the `topics` entry, the dependency-order rule for `schema.sql`, and an expanded PythonAnywhere deploy section including a pre-flight check for case-variant topic names.

Deliberately unchanged: `flashcards.topic` is still written (it is what `ai_agent`'s `cards_db` reads, and the rollback), no template or JS was touched, `/topics.json` keeps its shape, and URLs still key on the topic name.

Automated tests

Added / updated — 51 net new tests across seven files:

File	Change
`conftest.py`	New shared `FakeCardCursor` / `FakeCardConn` / `fake_card_db()`. The card write paths are no longer one query each, so a fake returning one canned row per `fetchone()` could not represent them; this one dispatches on the statement.
`test_apply_schema.py`	Fake database taught about data steps and probes; 11 new tests covering the probe contract, the dry-run tolerance,

	and an interrupted backfill. Table-count assertions replaced with dependency-order ones.
`test_apply_schema_db.py`	Hardcoded change counts replaced with named-step assertions (a total goes stale on the next migration); 6 new real-MySQL tests for the backfill.
`test_topics_table_db.py`	**New file** — 12 real-MySQL tests for the app-facing behaviour: creation, creator stability, canonical spelling, empty topics, case-insensitive matching, and account deletion.
`test_card_move.py`	Storage-layer tests moved to the shared fake; 7 new tests for get-or-create, canonical spelling, and the refusal paths.
`test_card_ownership.py`	Owner is no longer the last bound value (`topic_id` trails it) — position now derived from `utils.FLASHCARD_FIELDS`; 2 new tests that a topic's creator also comes from the session, never the entry.
`test_duplicates.py`, `test_action_logs.py`, `test_individual_cards.py`	A refused save creating no topic; the `TOPIC` log line; `get_topics()`'s new SQL shape and that a topic's creator never filters a view.

Suite result:

- `pytest` (offline default) — **599 passed, 27 skipped** (was 575 passed, 10 skipped; the extra skips are the new db-marked tests).
- `RUN_DB_ROUNDTRIP=1 pytest` — **626 passed, 0 failed.**

A baseline run of the unchanged suite against `main` in a throwaway worktree gave **575 passed / 0 failed**, so every one of the 22 failures this change first produced was a test pinned to an internal that moved, not a regression. Each was read before being changed.

Manual & browser verification

The migration, on a clone of the live local database (56 cards, 7 topics, 51 unowned cards) with edge cases the real data lacks injected: a case-variant topic (`EMOTIONS` alongside `emotions`), an empty-string topic, a NULL topic, and a topic whose earliest card is owned.

- `apply_schema.py --dry-run` on a pre-#207 database → reported 5 pending steps and **did not crash on the backfill probe**, which names a column that does not exist yet. Database unchanged afterwards.
- Apply → 5 steps applied. 0 cards left with a topic but no `topic_id`; 7 topic rows.
- Creator and age → each topic dated from its earliest card (e.g. `general` → 2026-07-09, not migration day); the injected `creator_probe` topic credited to user 7; topics whose earliest card was anonymous left with `NULL`.
- Case variant → `EMOTIONS` and `emotions` collapsed to one row, and every card's `topic` string rewritten to the surviving spelling (checked with `COLLATE utf8mb4_bin`).
- Topicless cards → `topic_id` stayed `NULL`; no topic named `''` was created.
- Second run → the backfill re-ran (an unlinked row was added deliberately) and left existing rows' ids and creators untouched. Third run → `nothing to do - 13 object(s) already in place`.

- Fresh empty database → `schema.sql` created all four tables with `topics` before `flashcards`, and every migration reported *already present*, including the backfill.

Behaviour, 25 assertions against the migrated clone: get-or-create; an existing topic reused without its creator changing; a duplicate save creating no topic; a blank topic staying `NULL`; a denied move leaving no stray topic; an anonymous mover refused; a case-variant move reading as *unchanged* rather than *denied*; an admin moving an unowned card; both columns agreeing after every write; an unknown topic page returning empty rather than erroring.

Account deletion (#165's flow, with a topic in it) → `resolve_user_cards()` then `delete_user()` succeeded for a user who had created a shared topic. The topic survived with `created_by_user_id` NULL, the other user's cards untouched, and the topic still listed. The delete-my-cards branch left an empty topic row, which `get_topics()` correctly omits.

Pages, rendered in-process against the migrated database (the keyword gate satisfied the way the suite does it, rather than typing the keyword into a browser):

- `/` → 200; topic tiles identical to before the migration, names and counts unchanged.
- `/flashcards/emotions` → 200, 4 cards. `/flashcards/EMOTIONS` → 200, the same 4 (case-insensitive).
- `/flashcards/no-such-topic` → 200 with "No flashcards saved under this topic yet" — still an empty page, not a 404.
- `/deck/IT` → 200. `/quiz/vocabulary` → 200, 3 answer inputs.
- `/topics.json` → 200, same `[[name, count], …]` shape, so the widget's browse panel and the move dialog's suggestions needed no change.

Cleanup: both scratch databases dropped; the baseline worktree and the `.env` copied into it removed. A pre-migration `mysqldump` of the local database was taken before applying.

Result

Pass. No failing tests in either mode. The migration is idempotent, safe to interrupt, and reports what it did.

Flagged for follow-up, not defects:

- **Phase 2 — `ai_agent` must learn to join `topics`** before `flashcards.topic` can go. Its `cards_db._normalize_row()` sniffs for the column with `_pick_column(columns, ["category", "topic", "tag", "part_of_speech", "pos"])`; drop `topic` and the next candidate that exists is **`pos`**, so Mykola would report every card's part of speech as its category with no error anywhere. Phase 3 then drops the column — and note `apply_schema.py` cannot yet express a `DROP`, because "the object exists" means *skip*, which is backwards for a step whose job is removal.
- **Empty topics have no page behaviour yet.** They can now exist; `get_topics()` deliberately still lists only topics with cards, which is what keeps this change invisible. Whichever ticket first *wants* an empty topic — a seeded curriculum (#203) or an image on an unfilled topic (#185) — decides what one looks like.
- **A correction to the ticket:** #207 asked for `move_card()`'s vanished-topic redirect to be removed. That was wrong and it stays — the topic still disappears from the list when its last card leaves, so landing back on its page would still read as a bug. Struck out on the issue.